

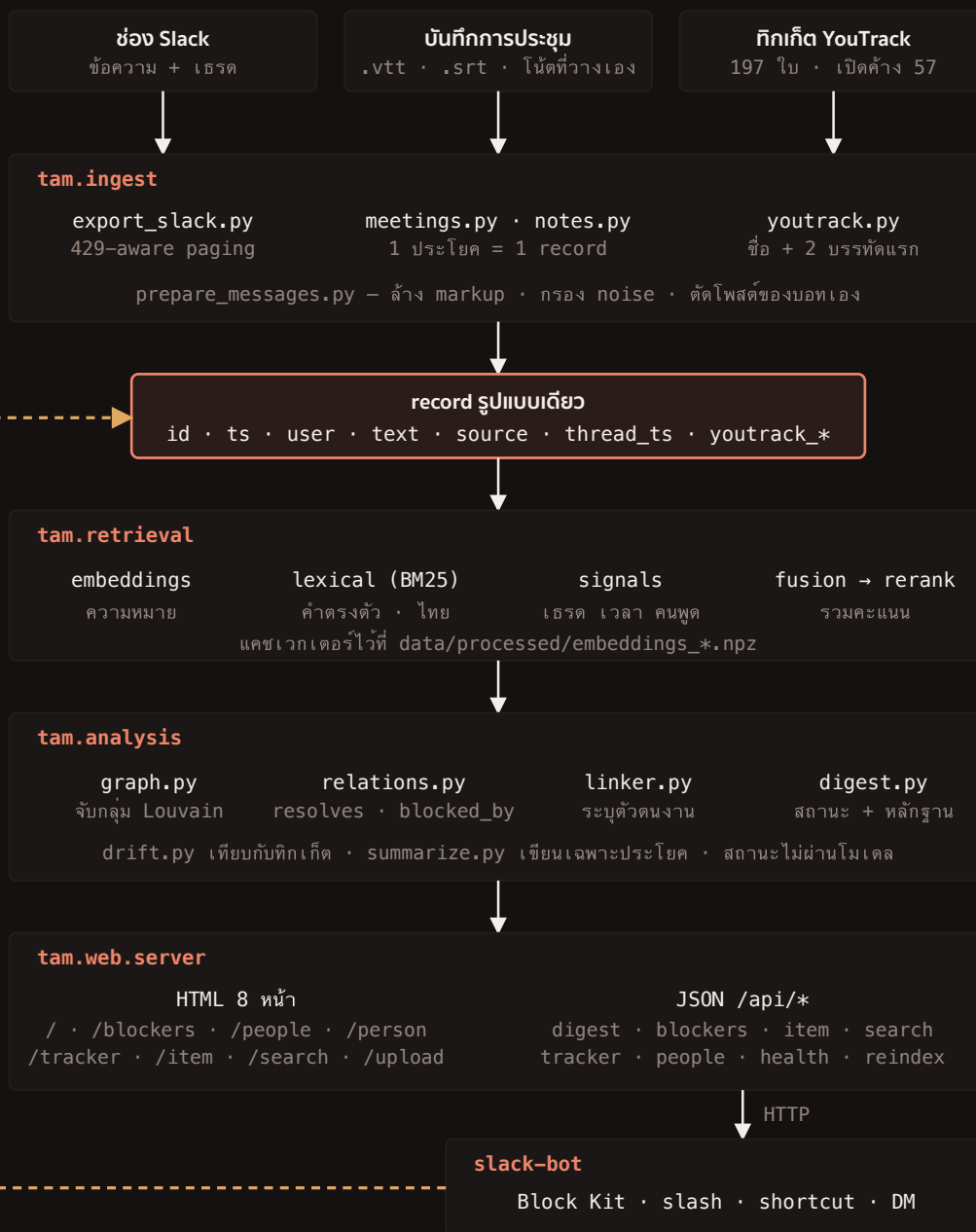
ข้อความหนึ่งข้อความ เดินทางไปเป็นการ์ดใน Slack ได้อย่างไร

แปดหัวข้อ หัวข้อละหนึ่งเรื่อง — เส้นทางข้อมูลทั้งเส้น, การให้คะแนน query หนึ่งครั้ง, ใครเป็นเจ้าของนิยาม “งานหนึ่งชิ้น”, การเขียนกลับเมื่อคนแก่, ด้านที่ทำให้ recall ตอบได้ว่า “ไม่เจอ”, หลักการที่มาจากการวัด, กราฟจากตัวเลขจริง และกฎที่ทำให้ทริกเกอร์เข้า corpus ได้โดยไม่กลืนงานที่คนคุยกัน แล้วปิดท้ายด้วย โครงโพลเดอร์ ทุกชื่อโมดูลในภาพคือไฟล์ที่มีอยู่จริงในรีโป

1 • เส้นทางทั้งเส้น

เรื่องสำคัญที่สุดของภาพนี้อยู่กลางภาพ: **Slack บันทึกการประชุม และทริกเกอร์ ถูกแปลงเป็น record รูปแบบเดียวกัน** ตั้งแต่ต้น ทุกขั้นหลังจากนั้นจึงไม่ต้องรู้ว่าข้อความมาจากไหน งานหนึ่งชิ้นข้ามสามแหล่งได้โดยไม่มีโค้ดพิเศษ — และทริกเกอร์มีกฎกันไม่ให้ตั้งตัวเป็นงานเอง (หัวข้อ 8)

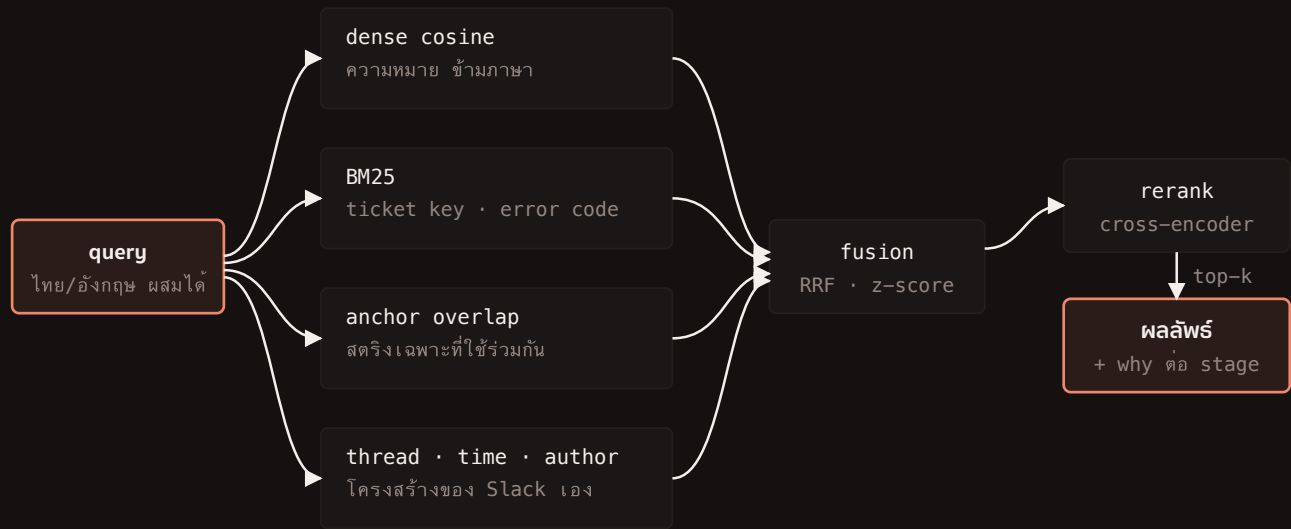
คนเก่า → link_overrides.json



ทุกชั้นอ่าน record รูปแบบเดียวกัน ทำให้การประชุมและทิกเก็ตไม่ต้องมี pipeline ที่สอง ทั้งสามแหล่งมาบรรจบที่แอสซัมเดียว แล้วทุกชั้นได้แอสซัมนั้นไม่รู้จักรำว่า “Slack” อีกเลย · เล่นประสึสัสมคือทางเดียวที่ข้อมูลไหลย้อนกลับ — คนแก็การผูก ticket ใน Slack แล้ว linker.py อ่านเป็นชั้นสูงสุดในรอบถัดไป (ภาพที่ 4)

2 · query หนึ่งครั้ง ได้คะแนนมาอย่างไร

cosine ตัวเดียวตอบไม่ครบ เพราะมันสมมาตร มองไม่เห็นสตริงตรงตัวอย่าง REV-1421 และไม่รู้วาสองข้อความอยู่เรรดเดียวกันห่างกันห้านาที แต่ละอย่างเป็นหลักฐานต่างชนิดกัน ภาพนี้คือลำดับที่

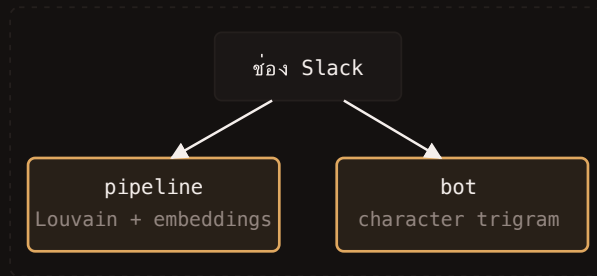


คะแนนแต่ละ stage ถูกเก็บไว้ ไม่ได้ทิ้ง จึงตอบได้ว่าอะไรทำให้ผลนี้ขึ้นมา — ในภาพที่ 5 คุณจะเห็นว่าคะแนนรวมหลัง RRF บอกไม่ได้ว่า “ไม่เจออะไรเลย” และแก้อย่างไร

3. ใครเป็นเจ้าของนิยาม “งานหนึ่งชิ้น”

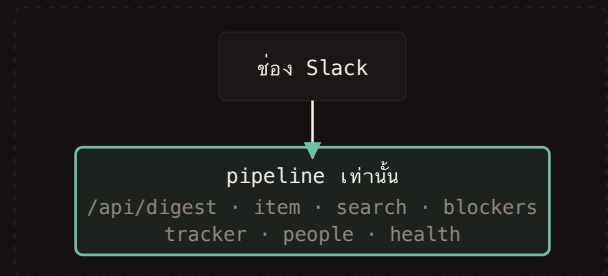
เดิมสองฝั่งอ่าน Slack เองและจับกลุ่มด้วยกฎต่างกัน ช่องเดียวกันจึงได้ work item ไม่เท่ากัน และคำตอบที่ทีมเห็นขึ้นอยู่กับการเปิดที่ไหนดึง TAM_API_URL แล้วเหลือเจ้าของเดียว

ก่อน - สองเจ้าของ



ได้ work item ไม่เท่ากัน

หลัง - เจ้าของเดียว



บอทเป็นตัวเรนเดอร์

มาจาก API

work item · สถานะ · หลักฐาน · อายุ
timeline · ข้อความ · ประโยคสรุป
recall (embeddings + BM25)
drift · ticket ที่เขียน · ใครอยู่กับเรื่องไหน

บอทเดิมเอง - pipeline ยังไม่มี

decision · จากที่คนกดบันทึก (เขียนลงไฟล์จริง)
standup draft · จำนวนจาก item ที่ได้นำ
การเสนอ Block Kit · ปุ่ม · modal

ไม่มี fallback

ถ้าตั้ง TAM_API_URL แล้ว pipeline ตอบไม่ได้ → บอกไม่สตาตัส ไม่แอบเสิร์ฟ fixture ที่หน้าตาเหมือนของจริง

ที่แบ่งไม่ได้แบ่งตามใจ แต่แบ่งตามว่าฝั่งไหนคำนวณจริง สิ่งที่เราไม่มีของเทียบใน pipeline ถ้าปล่อยว่างคือลูป
พีเจอร์ที่ทำงานอยู่ทั้ง จึงเติมจากแหล่งที่เป็นของจริงของมันเอง · drift เคยอยู่ขวามือในฐานะ “ยังไม่มีแหล่ง” — พอ
ต่อ YouTrack เข้ามาจริง มันก็ย้ายมาซ้าย drift.py จำนวนถูกรอบ แล้วออกทาง /api/tracker ทั้งบอกและ
หน้าเว็บอ่านชุดเดียวกัน

4 · คนแก่แล้วระบบจำ — วงเวียนกลับ

การจัดกลุ่มอัตโนมัติทำได้ และเมื่อคนแก่ ระบบต้องไม่เดากับรอบหน้า linker.py มีชั้น
override ที่ชนะทุกชั้นอยู่แล้ว และ load_overrides ของมันเขียนไว้ว่ารับ “the list-of-events
shape a UI would append to” — วงนี้จึงปิดที่จุดที่ถูกออกแบบเพื่อไว้



ชั้นของหลักฐานใน linker — บนสุดชนะ

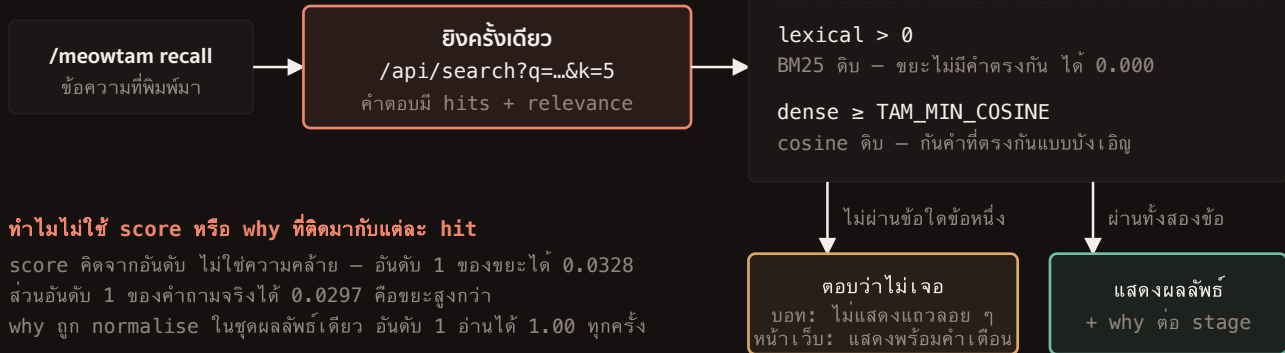
override explicit ticket key thread consensus cluster / unassigned

“คนแก่เอง” ถูกเก็บเป็นเหตุผลกำกับ ทำให้ override ที่ผิดตามกลับไปหาคนได้

ทดสอบครบวงแล้ว เขียน override ในรูปแบบที่บอกเขียน แล้วรัน linker.py ได้ผล REV-9999 · 1 msg ·
[override 1] พร้อมเหตุผล คนแก่เอง — ไม่ใช้การตอบบนกระดาน

5 · ด้านที่ทำให้ recall ตอบว่า “ไม่เจอ” ได้

คะแนนรวมคิดจากอันดับ ไม่ใช้ความคล้าย มันจึงบอกไม่ได้ว่า “ไม่เจออะไรเลย” — วัดบน corpus จริง
อันดับ 1 ของ qqzxxx wwwvvv jjjkkk zzzqqq ได้ 0.0328 ส่วนอันดับ 1 ของคำถาม
จริงได้ 0.0297 คือขยะได้คะแนนสูงกว่า และ why ของแต่ละแถวถูก normalise ในชุดผลลัพธ์ของ
ตัวเอง อันดับ 1 จึงอ่านได้ 1.00 ทั้งสองแบบ ตัวเลขที่เทียบข้าม query ได้จึงมีแค่สองตัวที่
/api/search ส่งมาให้ต่างหาก — relevance.lexical (BM25 ดิบ) กับ
relevance.dense (cosine ดิบ) — และด้านต้องใช้ทั้งคู่ ไม่ใช่ตัวใดตัวเดียว



ทำไมไม่ใช้ score หรือ why ที่ติดมากับแต่ละ hit
 score คิดจากอันดับ ไม่ใช่ความคล้าย — อันดับ 1 ของขยะได้ 0.0328
 ส่วนอันดับ 1 ของคำถามจริงได้ 0.0297 คือขยะสูงกว่า
 why ถูก normalise ในชุดผลลัพธ์เดียว อันดับ 1 อ่านได้ 1.00 ทุกครั้ง

วัดบน corpus จริงตอนนั้น 1,102 record (ตอนนี้ 1,301 - เกณฑ์ต้องวัดใหม่เมื่อ corpus เปลี่ยน) · ตัวเลขเพิ่มอยู่ใน docs/EXPERIME
 กลไกของด่าน

	ขยะที่กรองได้	query จริงที่เสียไป
cosine ≥ เกณฑ์ อย่างเดียว (กลไกเดิม)	0/5	0/12
z-score — top hit โดดจากกลุ่มโหม	ยังทับกัน	—
BM25 > 0 และ cosine ≥ เกณฑ์ (ตอนนี้)	4/5	0/12

cosine เดี่ยวแยกไม่ได้ทั้ง 5 โมเดลที่ลอง — max cosine เหนือ N เอกสารสูงขึ้นตาม N ยิ่งไงก็มีอันที่คล้าย
 ตัวที่ยังหลุดคืออักขระไทยซ้ำ ๆ (ๆ ๆ) ซึ่งมีในข้อความปกติจริง จึงมีค่าตรงกันจริง — เป็นขอบเขตของกลไก ไม่ใช่ตั้งค่าผิด

เปลี่ยนกลไก ไม่ใช่เปลี่ยนเลข เดิมเป็นสองคอล คอลแรกถาม cosine แล้วเทียบกับ floor ทีเดียว ๆ — เลิกเพราะวัดแล้วไม่ได้ผล ไม่ใช่เพราะเปลี่ยนคอล ตอนนี้เหลือคอลเดียวและตัดสินจากสองสัญญาณ ที่ server คิดมาให้ (cross-encoder ก็ลองเอามาเป็นด่านแล้วไม่ผ่านการวัด — เหตุผลอยู่ในคอมเมนต์ของ searchWithRelevance) วัดซ้ำที่หน้างานด้วย npm run check-api เพราะเกณฑ์เป็นคุณสมบัติของ corpus คู่กับโมเดล ไม่ใช่ค่าคงที่

6 · หลักการคิด — และการวัดที่ทำให้ได้หลักการนั้น

ห้าข้อนี้ไม่ได้ตั้งไว้ก่อนเริ่มเขียน ทุกข้อมาจากบักที่วัดเจอแล้วต้องกลับไปแก้กลไก ไม่ใช่แก้ตัวเลข — คอลัมน์ขวาคือเรื่องที่ทำให้ได้ข้อนั้นมา

หลักการ	เจอมาจากอะไร
ทุกคำตอบต้องชี้ข้อความที่เป็นหลักฐาน ถ้าอธิบายไม่ได้ว่ารู้มาจากไหน ก็ไม่ต้องพูด	สถานะทุกอันเก็บ evidence_id ของข้อความที่ทำให้สรุปแบบนั้น พอเก็บแล้วบักโฟลเอง — PROJ-157 ถูกมาร์คว่าปิดแล้วโดยอ้าง “added permission done krub” ซึ่งไม่มีคำว่า PROJ-157 อยู่เลย
กฎที่อธิบายได้ ชนโมเดลที่เตาถัง	สถานะมาจาก typed relation ที่ระบุ cue ได้ ไม่ได้มาจาก LLM เวลาพิดจึงชี้ได้ว่าพิดที่บสรกิดไหนแล้วก็ได้ — และแก้ไปแล้วสามครั้งด้วยวิธีนี้

หลักการ	เจอมาจากอะไร
ใช้ embedding หา ใช้กฎ ตัดสิน	
วัดสองทิศทาง ไม่ใช่ทิศทางง่าย เลขที่ขึ้นง่ายมักขึ้นเพราะ เหตุผลผิด	จะเพิ่ม resolution ของการจับกลุ่ม ต้องวัดทั้ง ยุบเกิน และ ผ่าเกิน พร้อมกัน ปรากฏว่าลดทั้งคู่ — ไม่มี tradeoff ที่กลัวไว้ (ดูกราฟ 7a)
เกี่ยวกับเรื่องเดียวกัน ไม่ใช่กับสิ่งที่อยู่ใกล้กัน อยู่คลัสเตอร์เดียวกัน ไม่ใช่ หลักฐาน	บักชนิดเดียวกันเกิดสามครั้ง: drift เกี่ยวกับหางของคลัสเตอร์ · คำประกาศว่าติดถูกยกเลิก ด้วย cue ของคนอื่น · ticket ถูกเทียบเพราะ record มันนั่งอยู่ในก้อนเดียวกัน ทั้งสามครั้งแก้ด้วยการบังคับว่าต้องเป็น เรื่องเดียวกัน
“ไม่รู้” ต้องต่างจาก “ไม่มี” ลิสต์ว่างเป็นคำตอบได้ ถ้า มันบอกว่าว่างเพราะอะไร	recall ปฏิเสธ query ที่ไม่มีคำร่วมกับ corpus แทนที่จะเดาห่าอัน · <code>/api/tracker</code> ส่ง <code>error</code> มาคู่กับลิสต์ว่างเมื่ออ่าน ticket ไม่ได้ และหน้าจอเขียนว่า “เรายังไม่รู้” ไม่ใช่ “ไม่มีงานค้าง”

หลักการข้อ 1 — ห่วงโซ่ที่ต้องต่อกันได้ทุกชั้น ไม่มีชั้นไหนขาด

การ์ดบนหน้าจอ
blocked since 18 ส.ค.
“mkt ต้องรอ ...”

อ้างอิง

work item
item_id · evidence_id
state_since

อ้างอิง

ข้อความเดินทาง
record id · user · ts
ข้อความจริงเต็ม ๆ

อ้างอิง

Slack permalink
เปิดอ่านในบริบทเดิม
642/691 สร้างได้

ที่ชั้นไหนก็ได้ ถ้าตัวเชื่อมหายไป ระบบต้องบอกว่าไม่รู้ — ห้ามข้ามไปแสดงผลลัพธ์ที่อธิบายที่มาไม่ได้

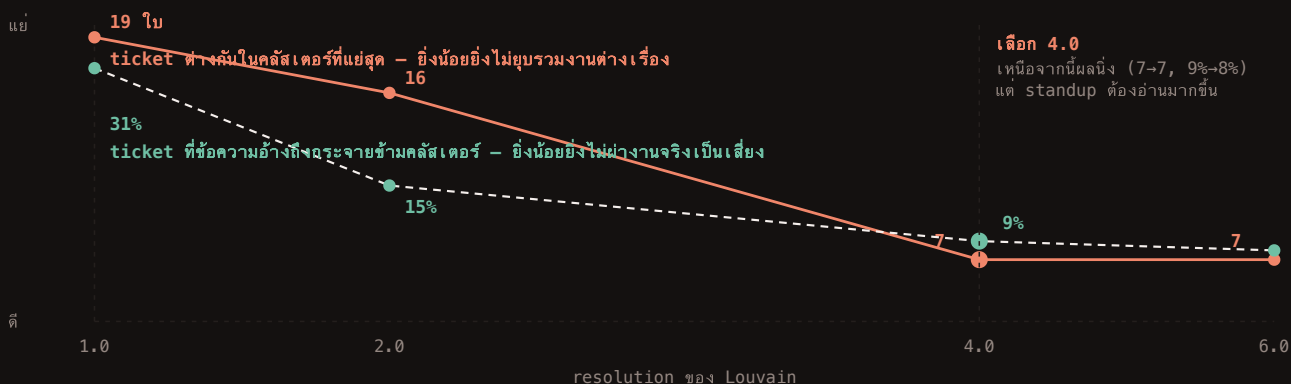
ข้อความจากที่ประชุมกับ ticket ไม่มี Slack permalink ตามธรรมชาติ — ระบบจึงพูดว่า “ไม่ได้มาจาก Slack”

ไม่ใช่พูดว่า “ลิงก์เสีย” และ ticket ได้ลิงก์ของตัวเองไปที่ YouTrack แทน

หลักฐานต้องต่อกันได้สี่ชั้น ทุกอย่างที่ระบบพูดถอยย้อนไปถึงข้อความที่คนพิมพ์จริงได้ — และเมื่อชั้นใดชั้นหนึ่งขาด
หน้าที่ของระบบคือบอกว่าขาด ไม่ใช่เดาให้เต็ม

7 · กราฟจากตัวเลขที่วัดจริง

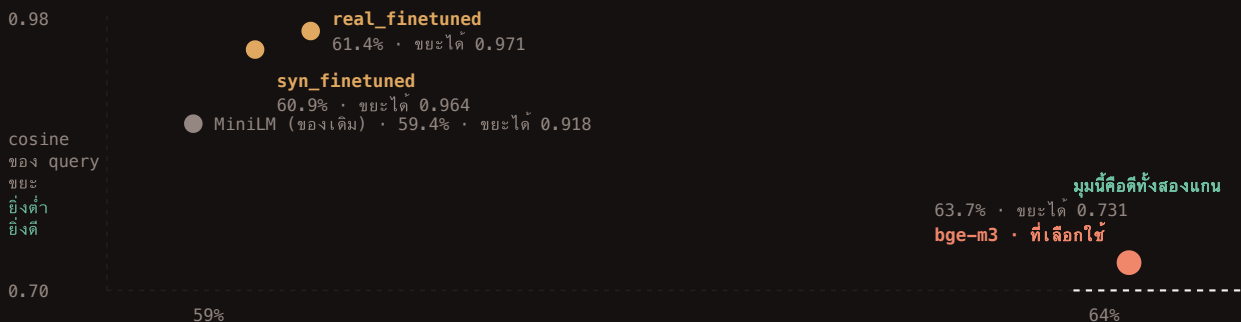
ทุกตัวเลขในหัวข้อนี้นี้มาจากการรันบน corpus จริง ไม่ใช่ตัวอย่าง — ตัวเลขปัจจุบันรวมบันทึกประชุมที่ mock ไว้
สองอันเพื่อทดสอบการรวมสามแหล่ง



ต้นทุนที่ต้องพูด: งานที่แสดงครอบคลุม record ลดจาก 90% เหลือ 85% เพราะก้อนเดี่ยวตกใต้เกณฑ์ – ยังค้นเจอ แต่ไม่ถูกเรียกว่างาน

เหตุผลที่ไม่ควรเพิ่ม resolution กลายเป็นเหตุผลที่ควรเพิ่ม ที่ค่าต่ำ ก้อนใหญ่ดูดข้อความตามช่องและเวลา ไม่ใช่ตามเรื่อง คนสองคนที่คุย ticket เดียวกัน ในสองที่จึงไปอยู่สองก้อน — **1.0** แย่กว่าทั้งสองทาง จึงไม่มี tradeoff ให้เลือก

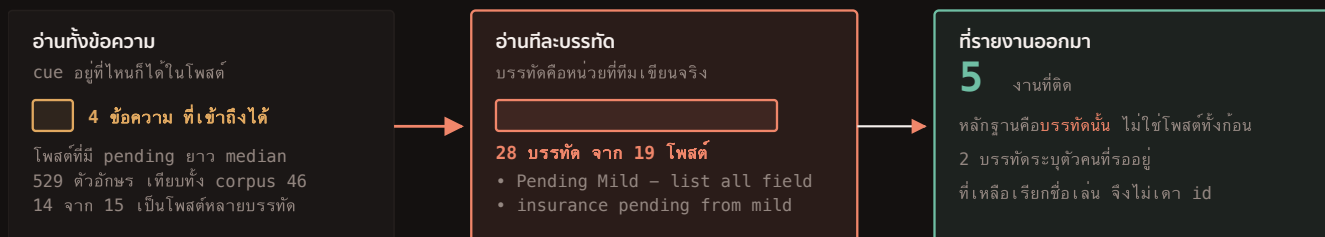
7b . โมเดลที่ทรนเองแพทั้งสองแกน ไม่ใช่แค่แกนเดียว



triplet accuracy บน thread ที่กันไว้ - ยิ่งขวยยิ่งดี
 เทรนเองแล้ว accuracy ขึ้นเล็กน้อยแค่ cosine ของ query ระยะเพื่อน = space บน ระยะจะคอบทก query อย่างมั่นใจ จึงไม่ใช้ทั้งสองตัว

ถ้าวัดแค่ความแม่นยำจะเลือกผิด ดูแค่ accuracy โมเดลที่เทรนเองชนะของเดิม แต่พอวัดว่ามีประสิทธิภาพ query ที่ไม่เกี่ยวข้องได้ไหมด้วย มันแยกว่าชัดเจน — และนั่นคือแกนที่ผู้ใช้รู้สึกจริง

7c · หน่วยของสัญญาณ - ไม่ใช่ลิสต์คำ



คำว่า 'pending' เป็น cue ระบุดังข้อความแล้ว blocked ขึ้นจาก 2 เป็น 3 - แต่หลักฐานของอันใหม่คือโพสต์ "Sprint 3 is officially end RECAP" ซึ่งเป็นสรุปสปริต ไม่ใช่การบอกว่าการคิด จึงถอนออก · ระบบที่ย่างสรุปสปริตเป็นหลักฐานว่ามีคนคิด น่าเชื่อถือน้อยลง ไม่ใช่มากขึ้น

คอบวคือระดับของสัญญาณ ทีมเขียน blocker เป็นบัสรกดหนึ่งในโพสที่ยาวกว่าข้อความปกติ 12 เท่า อ่านทั้งโพสจึงได้ cue ที่ถูกแต่อาจที่มาผิด — เปลี่ยนหน่วย ไม่ใช่เติมคำ

7d · ticket ที่เขียน — รูปแบบ ไม่ใช่ลิสต์

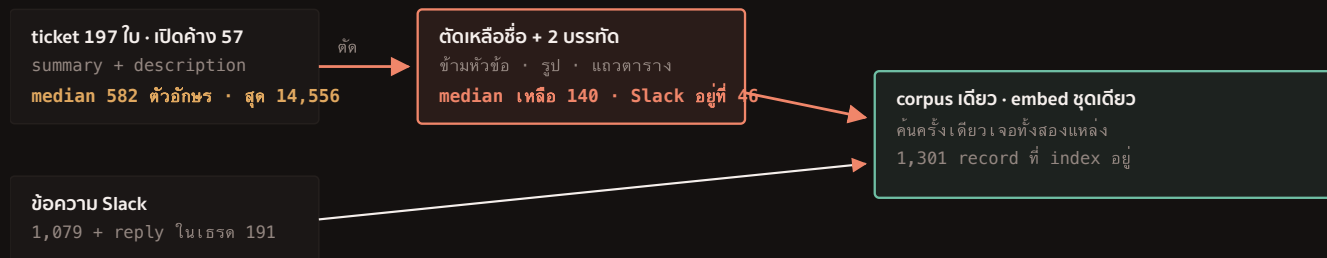


สองยอดคืองานสองก้อนที่ถูกดันเข้า Ready for test วันเดียวกันแต่ละก้อน แล้วไม่มีใครหยิบต่อ — 22 จาก 24 ใบอยู่สถานะเดียวกันนี้

เกณฑ์มาจากการกระจายของทีมเอง ไม่ใช่ตัวเลขกลม ๆ — และเลือกจุดที่ตกในช่องว่างธรรมชาติ เพื่อให้เลื่อนหนึ่งวันคำตอบไม่เปลี่ยน · สิ่งที่มีค่าคือรูปร่างของสองยอด ไม่ใช่จำนวน 24

8 · สองแหล่งกลายเป็น corpus เดียว — และกฎที่กันไม่ให้มันกลืนกัน

8 · ticket เข้า corpus แต่ห้ามตั้งตัวเป็นงานเอง



กฎเดียวกันก่อน ticket ล้วน

บทสนทนาในก่อนต้องไม่น้อยกว่า ticket ในก่อนนั้น มิฉะนั้นไม่นับเป็นงาน

วัดแล้ว: 43 จาก 196 ใบไปเกาะกับบทสนทนาจริงใน 19 งาน — ที่เหลือไม่ตั้งตัวเป็นงานเลย (0 งานที่มีแต่ ticket)

ก่อนแบบนี้จะโผล่เป็น “งานหนึ่งชิ้น” ชื่อ api, then, view ซึ่งไม่ใช่งาน มันคือ backlog

ทั้ง 196 ใบยังค้นเจอครบ และยังถูกเทียบครบ — ที่ถูกกันคือการโผล่บน digest เท่านั้น เพราะ /meowtam silent คุณเรื่องนี้ได้ดีกว่าอยู่แล้ว

เข้า corpus ได้ แต่สร้างงานเองไม่ได้ ticket เป็นส่วนขยายของงานที่คนคุยกัน ไม่ใช่ต้นกำเนิดของงาน — และ ticket ที่ไม่มีใครคุยถึงมีรายงานของตัวเองที่ทำได้ละเอียดกว่า

สองฝั่งเป็นคู่กัน อยู่ระดับเดียวกัน และเข้าแบบเดียวกัน — `cd pipeline` กับ `cd slack-bot`
แต่ละฝั่งมี `.env.example`, Slack app manifest และ `data/` ของตัวเอง

pipeline/

Python · เครื่องยนต์ + dashboard

`tam/` ingest · retrieval · analysis
evaluation · report · web

`data/` raw · processed · sample

`fixtures/` · `tests/` ตัวอย่าง API response · pytest

`models/` น้ำหนักที่ fine-tune (ไม่ขึ้น git)

`.env.example`

`slack-app-manifest.json`

slack-bot/

TypeScript · Bolt · Socket Mode

`src/` app · data · search · store
standups · tam-api · blocks/

`scripts/` export · ledger · preview · check-api

`data/` ledger.fixture (ขึ้น git) · ledger · deci

`tests/` node --test

`.env.example`

`slack-app-manifest.yaml`

`docs/` USER_MANUAL · DESIGN_BRIEF · concept · deck · [หน้านี้](#) — เอกสารเท่านั้น ไม่มีข้อมูลทดสอบปน

เดิม `tam/` อยู่ระดับ 0 แต่บอกอยู่ลึกสองชั้นใน `apps/meowtam/` และ `apps/` ห่อของไว้ตัวเดียว `deploy/` มีไฟล์เดียว — ทั้งสองอย่างถูกยุบทิ้ง